

BCSE355L

Computer Architecture Design

Fall Semester 2026-27

**Telemetry-Aware Multi-Agent Infrastructure-as-
Code Assurance System for Smart Manufacturing
Cloud Environments**

Name: Dhanush R

Reg No: 24BIT0178

Abstract

Infrastructure-as-Code (IaC) has emerged as a fundamental approach for automating cloud infrastructure provisioning by enabling infrastructure resources to be defined through machine-readable configuration files. With the increasing adoption of cloud-native technologies in smart-manufacturing and Industrial Internet of Things (IIoT) environments, ensuring the security, reliability, and correctness of IaC deployments has become increasingly challenging due to security misconfigurations, deployment failures, configuration drift, policy violations, and inconsistent infrastructure states. Existing research has demonstrated significant progress in IaC generation, security-aware validation, automated repair, deployment verification, configuration validation, and industrial automation using Large Language Models (LLMs). However, most existing solutions address isolated tasks, focus on specific IaC technologies or industrial components, and rely primarily on static analysis without integrating runtime cloud telemetry or comprehensive deployment validation.

This project proposes a **Telemetry-Aware Multi-Agent Infrastructure-as-Code Assurance System for Smart Manufacturing Cloud Environments**. The proposed framework employs specialized LLM-based agents to perform IaC parsing, syntax validation, security assessment, deployment verification, configuration drift detection, cost analysis, and explainable recommendation generation within a unified workflow. By combining static Infrastructure-as-Code validation with runtime telemetry collected from cloud monitoring services, the framework enables intelligent detection of infrastructure inconsistencies and provides automated remediation recommendations before and after deployment. The system supports multiple IaC technologies, including Terraform, AWS CloudFormation, AWS CDK, Kubernetes manifests, and AWS Serverless Application Model (SAM). Implemented using AWS services such as Amazon Bedrock, AWS Lambda, Amazon CloudWatch, Amazon S3, Amazon DynamoDB, Amazon API Gateway, Amazon Cognito, and AWS IAM, the proposed framework aims to improve deployment reliability, strengthen cloud security, reduce manual validation effort, and provide a scalable, explainable solution for intelligent Infrastructure-as-Code assurance in modern smart-manufacturing environments.

Literature Survey

2.1 Introduction

Infrastructure-as-Code (IaC) has become a fundamental approach for automating cloud infrastructure provisioning by enabling infrastructure resources to be defined through machine-readable configuration files. With the rapid adoption of cloud-native technologies in smart-manufacturing environments, managing secure, scalable, and reliable cloud infrastructure has become increasingly challenging. Recent advances in Large Language Models (LLMs) have created new opportunities for automating Infrastructure-as-Code generation, validation, security analysis, configuration repair, deployment verification, and intelligent decision support. The reviewed studies demonstrate significant progress in these areas while also highlighting several technical limitations that motivate the proposed Telemetry-Aware Multi-Agent Infrastructure-as-Code Assurance System for Smart Manufacturing Cloud Environments.

2.2 Evolution of Research

Early research primarily focused on improving Infrastructure-as-Code generation through retrieval-augmented generation, knowledge injection, and benchmark-based evaluation for Terraform and cloud deployment templates. Later studies introduced security-aware code generation, automated Infrastructure-as-Code repair, Kubernetes misconfiguration detection, enterprise DevSecOps validation, and deployment-oriented evaluation to improve infrastructure reliability and security. More recent research has expanded toward multi-agent reasoning, configuration drift detection, industrial configuration validation, PLC code generation and verification, Industrial Internet of Things (IIoT) security, and intelligent Industrial Control System (ICS) generation. These developments demonstrate the growing role of LLMs in cloud infrastructure and industrial automation while revealing the need for an integrated framework capable of combining validation, deployment verification, runtime telemetry, security assessment, and smart-manufacturing support.

2.3 Literature Review

The reviewed studies collectively demonstrate significant progress in applying Large Language Models (LLMs) to Infrastructure-as-Code (IaC), cloud infrastructure management, and smart-manufacturing systems. Existing research has improved IaC generation through retrieval-augmented generation, knowledge injection, security-aware fine-tuning, automated repair, deployment verification, and multi-agent reasoning. Recent studies have further extended LLM applications to configuration validation, PLC code generation and verification, Industrial IoT security, and Industrial Control System (ICS) automation. While these approaches achieve high accuracy within their respective domains, most focus on a single IaC technology, industrial component, or isolated validation task. Common challenges include limited multi-format support, dependence on static analysis, lack of runtime

telemetry integration, limited deployment verification, and minimal coordination between validation, security assessment, configuration drift detection, and recommendation generation. These limitations motivate the proposed **Telemetry-Aware Multi-Agent Infrastructure-as-Code Assurance System for Smart Manufacturing Cloud Environments**, which integrates these capabilities into a unified intelligent framework.

2.4 Literature Summary

Overall, the reviewed literature demonstrates substantial progress in Infrastructure-as-Code generation, security-aware validation, automated repair, deployment verification, configuration drift detection, industrial configuration validation, PLC verification, and Industrial IoT security using Large Language Models. Despite these advances, most existing approaches address isolated stages of the Infrastructure-as-Code lifecycle or focus on a single technology, such as Terraform, Kubernetes, PLCs, or Industrial Control Systems. A unified framework capable of supporting multiple Infrastructure-as-Code technologies while integrating deployment verification, runtime cloud telemetry, security assessment, configuration drift detection, explainable recommendations, and smart-manufacturing support remains largely unexplored. These recurring research gaps form the foundation for the proposed **Telemetry-Aware Multi-Agent Infrastructure-as-Code Assurance System for Smart Manufacturing Cloud Environments**.

Paper Title	Method	Dataset	Advantages	Limitations	Research Gap
laC Generation with LLMs: An Error Taxonomy and A Study on Configuration Knowledge Injection	Evaluates LLM-based Terraform code generation across Naive RAG, Graph RAG, and three graph enhancements (GR-OptMatch, GR-LLMSum, GR-Ref) using cloud emulation (LocalStack) and automated validation.	laC-Eval benchmark enhanced with LocalStack, comprising 458 Terraform generation prompts and official AWS Provider v5.90.0 documentation.	Boosts technical validation success from 37.2% to 84.2% and overall success from 27.1% to 62.6%. Fully local execution via LocalStack eliminates AWS account costs, network latency, and accidental resource provisioning. Provides a detailed two-dimensional taxonomy of 19 error types.	Intent validation plateaus at 72%-76% due to the 'Correctness-Congruence Gap'. Graph RAG with full reference modeling (GR-Ref) causes cognitive overload and degrades performance on simpler prompts. Restricted exclusively to Terraform configurations for AWS.	Existing knowledge-injection approaches focus narrowly on single-provider (AWS) Terraform syntax, leaving an intent-alignment gap (the Correctness-Congruence Gap) and no mechanism to optimize context retrieval across the heterogeneous, multi-format laC stack this project targets.
GENSIAC: Toward Security-Aware Infrastructure-as-Code Generation with Large Language Models	Instruction fine-tuning of base LLMs (CodeLlama 7B/13B) using LoRA (Low-Rank Adaptation) and 8-bit quantization to inject security-weakness awareness into generation and inspection workflows.	GENSIAC Dataset: 22K data points (11K code generation, 11K code inspection pairs covering Ansible, Chef, Puppet across 1,413 repos) + 148 test scripts (229 security flaws).	Raises security-weakness detection F1-score from ~0.30 to 0.858 (outperforming GPT-4's 0.592). Improves syntactical and functional correctness scores to 1.000 and 0.979. Resource-efficient fine-tuning requiring under 10 hours per epoch on an A100 GPU.	Does not cover all security weakness categories (e.g., CWE-276) due to reliance on upstream static-detection tools. Cross-language transfer is weaker on Ansible than on Chef/Puppet. Only tested with LoRA; no runtime cloud validation.	Security-aware fine-tuning here still depends on static scanners for training labels and stops at code inspection, with no live deployment verification — a gap this project's security-assessment agent closes by pairing static checks with runtime telemetry.
INFRAFIX: Technology-Agnostic Repair of Infrastructure as Code	A polyglot Automated Program Repair (APR) framework using an extended object-oriented Intermediate Representation (IR), a normalization database, system-call-based state inference, and an SMT-based repair engine.	Curated Polyglot Dataset: 12,358 Ansible, 4,868 Puppet, 3,568 Chef, 1,300 Terraform, and 13 Tortoise scripts, yielding 254,288 repair scenarios.	Technology-agnostic architecture operating across Ansible, Puppet, Chef, and Terraform simultaneously. High overall repair success rate of 95.7% across 254,288 scenarios. Includes an automated mutation-based repair-scenario generator.	Fails when resource names cannot be statically resolved into strings or are defined in uninstantiated classes/functions. Struggles with undefined-variable references and shared-variable side effects. Requires separate state-inference modules for non-filesystem resources.	Symbolic, constraint-driven repair cannot dynamically resolve runtime variables or cloud-resource dependencies without hand-written rules — motivating this project's use of LLM-agent reasoning alongside static repair for dynamically evolving smart-manufacturing infrastructure.
GenKubeSec: LLM-Based Kubernetes Misconfiguration Detection, Localization, Reasoning, and Remediation	End-to-end framework pairing GenKubeDetect (CodeT5+ fine-tuned with LoRA for multi-label classification) with GenKubeResolve (Mistral-7B guided by few-shot prompts for localization, reasoning, and remediation).	Unified Misconfiguration Index (UMI) with 169 misconfig classes + 276,520 Kubernetes Configuration Files (KCFs) from 'The Stack' + 700MB domain text.	High detection precision (0.990) and recall (0.999) versus static rule-based tools. Provides 100% expert-validated line-level localization, reasoning, and remediation. Adapts to new misconfigurations from just 1-5 labeled examples.	Constrained by CodeT5+'s 512-token input limit, leaving 17% of longer KCFs unanalyzed. High false-positive rates during few-shot adaptation on similar categories (e.g., RBAC rules). Qualitative evaluation limited to 100 false positives and 30 resolutions.	Token-limited, Kubernetes-only detection models cannot scale to the multi-service AWS architectures (EC2, S3, IAM, Lambda) this project must validate, nor do they coordinate with agents handling other laC formats in one pipeline.
Practical Integration of Large Language Models into Enterprise CI/CD Pipelines for Security Policy Validation: An Industry-Focused Evaluation	Enterprise DevSecOps framework augmenting static scanners (OPA/Checkov) with an LLM Ensemble (GPT-4, domain fine-tuned models, and ReAct input-sanitization filtering) inside Jenkins/Bamboo CI/CD pipelines.	Synthetic laC Suite: 500 cases (300 K8s, 200 IAM/Terraform) + real-world validation on 318 enterprise Helm charts (~47k LOC).	Ensemble method achieves high classification accuracy (F1 = 0.95) at low latency (3.1s). ReAct input filtering cuts prompt-injection attack success from 60% down to 7%-12%. Reduces manual security-review time by 60% and monthly rule maintenance by 70%.	Ensemble approach requires 3x computational resources, raising operational cost. Generates false alarms (~23% hallucination rate) and is over-confident on ambiguous code. Evaluation restricted to mid-scale, largely synthetic workloads.	High-cost ensemble validation with a ~23% hallucination rate highlights the need for a cost-optimized, coordinated multi-agent pipeline that pairs automated validation with real-time cloud telemetry — exactly the gap this project's Lambda-orchestrated agent architecture is designed to fill.
SWE-InfraBench: Evaluating Language Models on Cloud Infrastructure Code	Introduces SWE-InfraBench, a benchmark for evaluating LLMs on incremental Infrastructure-as-Code (laC) editing tasks in AWS CDK repositories using natural-language instructions and automated unit tests.	100 instruction-based tasks collected from 34 real-world AWS CDK repositories.	Focuses on realistic enterprise workflows; supports imperative laC editing; uses automated validation against real-world repositories.	Restricted mainly to AWS CDK. Does not fully analyze deployment reliability, security compliance, or production-scale environments.	A single-format (CDK) editing benchmark cannot verify cross-format deployment reliability or security compliance — this project's framework must jointly validate CDK alongside Terraform, CloudFormation, Kubernetes, and SAM within one workflow.
Multi-laC-Eval: Benchmarking Cloud Infrastructure as Code Across Multiple Formats	Proposes Multi-laC-Bench, a benchmark for laC generation and mutation across CloudFormation, Terraform, and CDK using natural-language requests.	Synthetic triplets consisting of an initial template, a modification request, and an updated template generated from public repositories.	Covers multiple laC languages; supports both generation and mutation; introduces standardized cross-format evaluation metrics.	Relies heavily on synthetic data and semantic evaluation via LLM judges and static-analysis tools rather than live deployments.	Multi-format coverage here is evaluated only on synthetic templates with no deployment-time or security feedback — this project extends multi-format validation with live AWS deployment verification and runtime telemetry, which synthetic benchmarks cannot provide.
Deployability-Centric Infrastructure-as-Code Generation: Fail, Learn, Refine, and Succeed through LLM-Empowered DevOps Simulation	Introduces laCGen, an iterative framework that refines generated templates using format verification, syntax checking, and live deployment feedback.	DPIaC-Eval benchmark containing 153 real-world scenarios across 58 AWS services.	Moves beyond syntax by emphasizing deployability; mirrors DevOps workflows; incorporates iterative feedback and human guidance.	Requires repeated deployment attempts and human intervention, increasing cost and reducing scalability. User-intent alignment and security compliance remain weak.	Iterative, human-in-the-loop refinement does not scale automatically and leaves security compliance unaddressed — this project's multi-agent design must automate refinement while adding a dedicated security-assessment stage that laCGen lacks.

Paper Title	Method	Dataset	Advantages	Limitations	Research Gap
RIVA: Leveraging LLM Agents for Reliable Configuration Drift Detection	Proposes RIVA, a multi-agent architecture with verifier and tool-generation agents that collaborate through cross-validation and tool-history tracking.	Evaluated on the AIOpsLab benchmark containing infrastructure localization and drift-detection tasks.	Improves robustness against faulty tools and enhances configuration-drift-detection accuracy.	Focuses mainly on configuration-drift detection and is evaluated only in benchmark settings, not real cloud deployments.	Drift detection is treated as an isolated, benchmark-only capability rather than one stage in a coordinated pipeline — this project integrates drift detection with generation, security assessment, and cost analysis using real CloudWatch/telemetry data rather than a static benchmark.
TerraProbe: A Layered-Oracle Framework for Detecting Deceptive Fixes in LLM-Assisted Terraform Security Repair	Introduces TerraProbe, a five-layer oracle framework for evaluating LLM-generated Terraform security repairs.	288 first-pass repairs generated across 68 real-world Terraform modules and 28 injected-defect modules.	Evaluates planning validity and security intent in addition to scanner results; includes statistical analysis and multi-model comparisons.	Limited to Terraform security repair and emphasizes evaluation rather than automatic secure-repair generation.	Rigorous evaluation of 'deceptive fixes' is confined to Terraform and stops short of generating the repair itself — this project needs equivalent deception-aware verification generalized across all supported IaC formats, feeding directly into automated remediation.
Large Language Models as Configuration Validators	Develops a generic LLM-based configuration validation pipeline and empirically evaluates multiple LLMs (e.g., GPT-4) on their ability to detect misconfigurations by prompting them with configuration values and system context.	Real-world configuration files and historical misconfiguration issues drawn from open-source systems (e.g., HDFS, HBase, Spark) used in prior machine-learning-based configuration validation research.	First systematic feasibility study proving LLMs can validate configurations without large-scale labeled field data or system-specific trained models, overcoming key limitations of ML-based validators.	Evaluated only on general-purpose software systems, not industrial or manufacturing control configurations; no retrieval grounding, so accuracy depends purely on the LLM's internal knowledge of each system.	Unclear whether this validation paradigm transfers to real-time, safety-critical manufacturing configurations (PLC/SCADA), where a missed misconfiguration causes physical, not just software, failure.
Agents4PLC: Automating Closed-Loop PLC Code Generation and Verification in Industrial Control Systems Using LLM-Based Agents	Multi-agent LLM framework combining Retrieval-Augmented Generation (RAG), advanced prompt engineering, and Chain-of-Thought reasoning to generate and formally verify PLC code end-to-end.	A purpose-built benchmark of natural-language PLC requirements paired with human-written, formally verified reference PLC code and specifications.	Demonstrates that RAG-enhanced multi-agent verification substantially outperforms single-shot LLM code generation for industrial control code, with measurable correctness gains.	Focused specifically on PLC/IEC-61131 code; does not extend to other manufacturing infrastructure layers such as SCADA networks, MES configuration, or edge orchestration.	No unified framework validates infrastructure code across the full manufacturing stack (PLC + SCADA + MES + edge) rather than treating PLC code generation in isolation.
LLM4PLC: Harnessing Large Language Models for Verifiable Programming of PLCs in Industrial Control Systems	A user-guided iterative pipeline combining LLM code generation with external verification tools (grammar checkers, compilers, SMV model checkers) and LoRA fine-tuning to progressively correct PLC program errors.	Validated on a FischerTechnik Manufacturing TestBed (MFTB), a physical smart-manufacturing testbed, alongside benchmark industrial control programming tasks.	Shows a clear, measurable evolution from structurally flawed to formally verifiable PLC code; real hardware validation on an actual manufacturing testbed strengthens practical relevance.	Still requires iterative human feedback in the loop, limiting full automation; depends on external formal verification tools that must be manually configured per PLC dialect.	No fully autonomous, continuously learning validation loop yet removes the need for repeated human feedback across diverse manufacturing hardware and PLC dialects.
Application of Large Language Models in the Protection of Industrial IoT Systems for Critical Infrastructure	Analytical study examining industrial automation system architecture and IIoT cyber-threat scenario classes, and assessing how LLMs could support monitoring, incident analysis, and compliance automation across IT/OT layers.	Not an empirical dataset-driven study; draws on architectural analysis of industrial automation systems and existing IIoT threat taxonomies and standards (e.g., ISO/IEC 62443, NIS2).	Provides a structured view of IT/OT convergence risks and positions LLMs as a tool for automating compliance and threat analysis specific to critical industrial infrastructure.	Conceptual and analytical rather than an implemented, empirically validated system; does not perform configuration or code-level validation directly.	Leaves an operational gap between "LLMs could help secure IIoT" and an actual, implemented, validated IaC/configuration framework for manufacturing environments.
From Words to Wires: Toward Rapid ICS Cyber-Range Construction Using LLMs	A lightweight LLM-integrated toolchain (Words2Wires) that converts plain-language descriptions into runnable ICS simulation artifacts — control scripts, simulation code, and OPC-UA tag maps — executed directly.	Example ICS scenarios (e.g., tank systems, traffic-light systems) used to demonstrate rapid testbed generation, with execution benchmarked at under 90 seconds per instance.	Demonstrates that LLMs can generate functioning, network-visible ICS infrastructure directly from natural language, dramatically cutting testbed setup time and vendor-specific tooling overhead.	Generated artefacts are executed "without manual edits or formal validation" — meaning the LLM-generated infrastructure code is never checked for correctness or security before running.	By the authors' own admission, no validation layer exists for LLM-generated industrial infrastructure code — precisely the gap your proposed framework is designed to fill.

Research Gap Analysis

The reviewed literature demonstrates significant progress in applying Large Language Models (LLMs) to Infrastructure-as-Code (IaC) generation, security validation, automated repair, configuration validation, industrial control systems, and smart manufacturing. Existing studies have introduced effective approaches for Terraform generation, security-aware IaC development, technology-agnostic repair, Kubernetes misconfiguration detection, enterprise DevSecOps validation, Infrastructure-as-Code benchmarking, PLC code generation and verification, Industrial IoT security, and intelligent Industrial Control System (ICS) generation. Despite these advances, the research remains fragmented, with most solutions addressing only specific stages of the Infrastructure-as-Code lifecycle or individual industrial domains rather than providing a unified validation framework.

A major limitation observed across the literature is the lack of **end-to-end, multi-format Infrastructure-as-Code validation**. Several studies are restricted to a single IaC technology such as Terraform, Kubernetes, or AWS CDK, while PLC-focused research concentrates only on industrial controller programming. Although these approaches achieve high accuracy within their respective domains, they cannot jointly validate heterogeneous cloud infrastructures consisting of Terraform, CloudFormation, Kubernetes, AWS CDK, Serverless templates, and industrial automation components within a single intelligent workflow. This limitation reduces their applicability in modern smart-manufacturing environments where cloud infrastructure and operational technology coexist.

Another recurring limitation is the heavy dependence on **static analysis or offline evaluation**. Most frameworks perform syntax checking, security scanning, benchmark evaluation, or formal verification before deployment, while providing limited support for monitoring deployed infrastructure. Industrial IoT studies remain largely conceptual, and ICS-generation frameworks prioritize rapid deployment without validating the correctness or security of generated configurations. Consequently, existing approaches fail to correlate Infrastructure-as-Code definitions with real-time operational information such as deployment logs, runtime telemetry, cloud monitoring metrics, and configuration drift, reducing their effectiveness in dynamic cloud-enabled manufacturing systems.

The literature also reveals that current solutions generally address **isolated tasks** such as code generation, security assessment, automated repair, PLC verification, or compliance analysis independently. None of the reviewed studies integrates intelligent parsing, security assessment, deployment verification, configuration drift detection, runtime telemetry analysis, cost evaluation, and explainable recommendation generation through a coordinated multi-agent architecture. Furthermore, several frameworks require human intervention, rely on domain-specific knowledge bases, or depend on external verification tools, limiting automation and scalability in large-scale industrial cloud environments.

Therefore, there remains a clear need for a **Telemetry-Aware Multi-Agent Infrastructure-as-Code Assurance Framework** capable of supporting multiple Infrastructure-as-Code technologies while integrating static validation, deployment verification, runtime cloud telemetry, configuration drift detection, security assessment, and explainable LLM-based recommendations within a unified workflow. Such a framework would bridge the gap between cloud-native Infrastructure-as-Code management and smart-manufacturing environments, providing secure, scalable, and intelligent infrastructure assurance across both Information Technology (IT) and Operational Technology (OT) ecosystems.

Project Objectives

1. **Develop** an intelligent, multi-agent Infrastructure-as-Code (IaC) validation framework that supports multiple IaC technologies such as CloudFormation, Terraform, AWS CDK, Kubernetes, and AWS Serverless Application Model (SAM) for smart-manufacturing cloud environments.
2. **Improve** the deployment success of LLM-generated or LLM-repaired Infrastructure-as-Code templates by incorporating automated validation, deployment verification, and intelligent feedback mechanisms.
3. **Enhance** the detection and mitigation of security misconfigurations, including insecure IAM policies, network configurations, and cloud resource settings, using LLM-assisted validation and automated remediation.
4. **Detect** and report configuration drift by correlating Infrastructure-as-Code definitions with runtime cloud telemetry, deployment logs, and resource states to ensure infrastructure consistency.
5. **Provide** explainable and cost-aware recommendations that improve the security, reliability, maintainability, and operational efficiency of Infrastructure-as-Code deployments while reducing manual validation effort.

Novelty Summary

The proposed **Telemetry-Aware Multi-Agent Infrastructure-as-Code Assurance System for Smart Manufacturing Cloud Environments** introduces an intelligent framework that integrates Infrastructure-as-Code validation, security assessment, deployment verification, configuration drift detection, and runtime telemetry analysis within a single coordinated workflow. Unlike existing studies that focus on isolated tasks such as Terraform generation, Kubernetes validation, PLC verification, or Industrial IoT security, the proposed framework provides a comprehensive, cloud-native solution for validating Infrastructure-as-Code across smart-manufacturing environments.

Novel Features

1. Unified Multi-Agent Validation Framework

- Integrates specialized LLM agents for IaC parsing, syntax validation, security assessment, deployment verification, configuration drift detection, cost analysis, and recommendation generation within one intelligent workflow.

2. Support for Multiple Infrastructure-as-Code Technologies

- Validates Terraform, AWS CloudFormation, AWS CDK, Kubernetes manifests, and AWS Serverless Application Model (SAM), overcoming the single-platform limitations of existing research.

3. Telemetry-Aware Infrastructure Validation

- Combines static Infrastructure-as-Code analysis with runtime telemetry collected from AWS CloudWatch, deployment logs, and cloud resource monitoring to improve validation accuracy and detect configuration drift.

4. Cloud-Native AWS Architecture

- Leverages AWS services including Amazon Bedrock, AWS Lambda, Amazon CloudWatch, Amazon S3, Amazon API Gateway, Amazon Cognito, Amazon DynamoDB, Amazon RDS, Amazon SNS, and AWS IAM to provide a scalable, secure, and serverless validation platform.

5. Enhanced Security and Automation

- Performs automated detection of security misconfigurations, validates deployment readiness, and generates explainable remediation recommendations, significantly reducing manual review effort.

6. Improved Validation Accuracy and Reliability

- Integrates intelligent reasoning with deployment verification and runtime monitoring to provide more reliable Infrastructure-as-Code validation than approaches based solely on static analysis or benchmark evaluation.

7. Scalable Smart-Manufacturing Deployment

- The modular architecture enables seamless integration with cloud-native manufacturing systems and can be extended to additional Infrastructure-as-Code technologies, industrial applications, and enterprise cloud environments with minimal modification.

Architecture Diagrams

Diagram 1- AWS Cloud Architecture

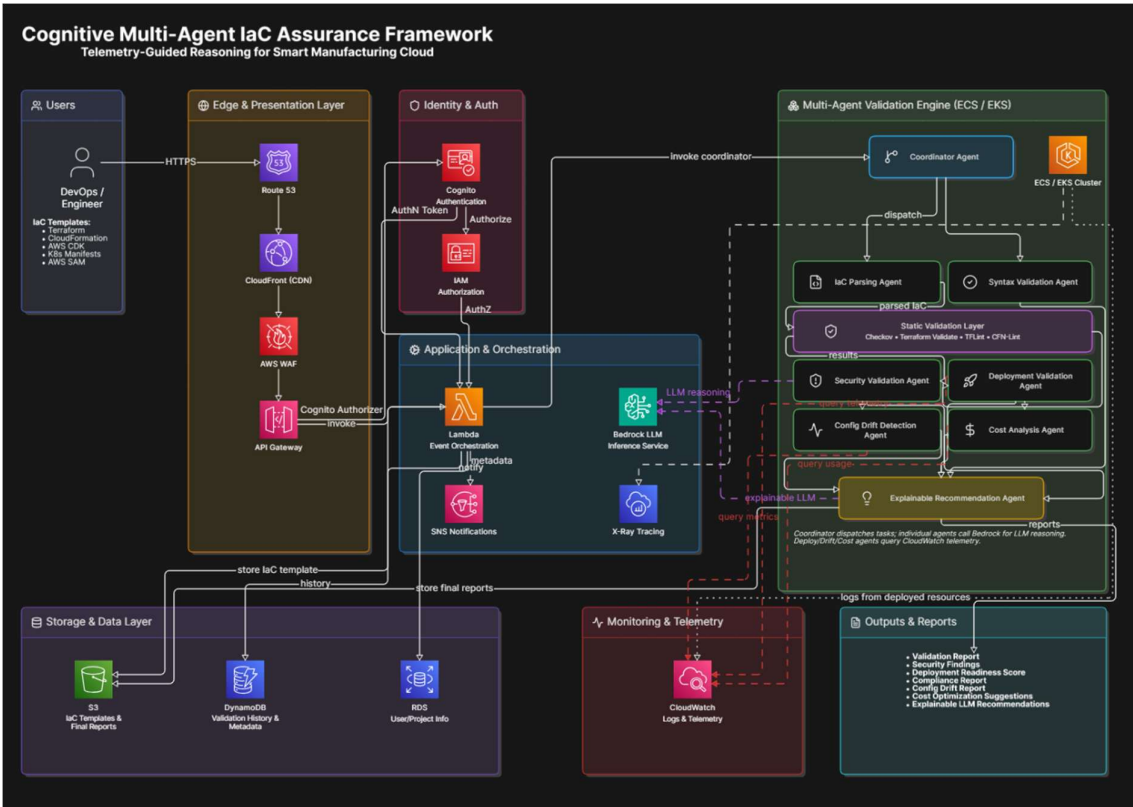
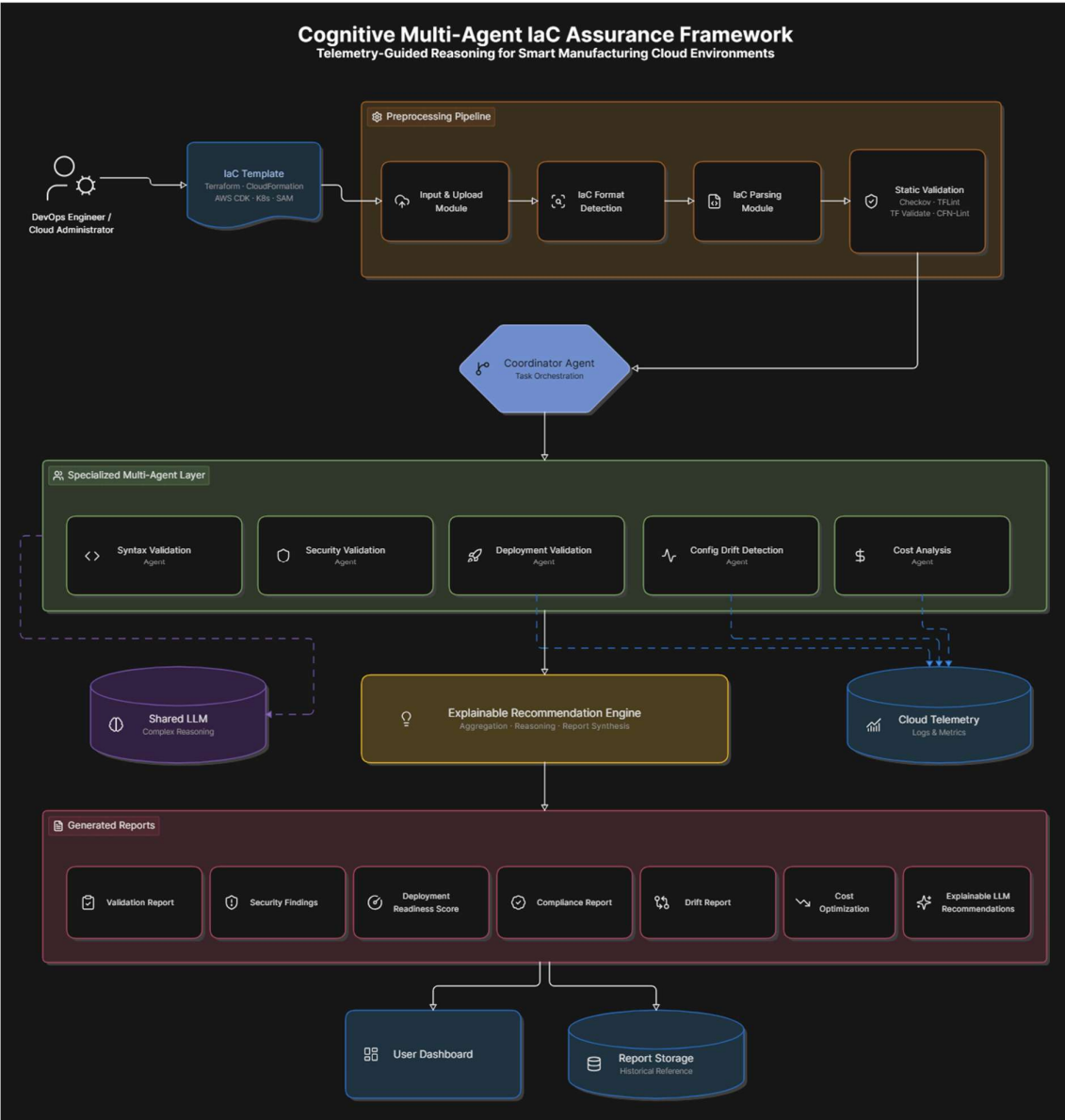


Diagram 2 - Complete System Architecture



Datasets

1. Dataset Name: IaC-Eval

- Source: Georgia Tech (NeurIPS 2024 Datasets & Benchmarks Track), hosted on Hugging Face / GitHub
- URL: <https://github.com/autoiac-project/iac-eval>
- Size: 458 human-curated Infrastructure-as-Code questions, ranging from simple to complex [GitHub](#)
- Number of Records: 458 scenarios
- Number of Features: Each record includes a natural-language problem description, an infrastructure-intent specification, and reference correctness checks (a natural language IaC problem description and an infrastructure intent specification used to verify if the generated program conforms to the user's intent)
- Data Type: Text (natural language) + Terraform HCL code
- License: Dataset released under Creative Commons Attribution 4.0; accompanying code under MIT license
- Purpose of Using the Dataset: Benchmark the IaC-generation and syntax-validation agent's ability to produce/validate correct, deployable Terraform code against known-correct infrastructure intent
- Preprocessing Required: Parse HCL into structured resource graphs; align each scenario's intent specification with your framework's internal validation schema; filter by target AWS services relevant to smart-manufacturing use cases

2. Dataset Name: Multi-IaC-Bench

- Source: arXiv preprint (2025), synthetic generation pipeline over public GitHub templates
- URL: <https://arxiv.org/abs/2509.05303>
- Size: Triplet-structured dataset spanning three IaC formats
- Number of Records: Multiple template–request–updated-template triplets consisting of initial IaC templates, natural language modification requests, and corresponding updated templates, created through a synthetic data generation pipeline with rigorous validation [arXiv](#)
- Number of Features: 3 core fields per record (original template, NL modification request, resulting template) plus metadata (format type, provider)

- Data Type: Text (CloudFormation, Terraform, and CDK code) + natural language
- License: Open academic release (check repository for specific terms at time of use)
- Purpose of Using the Dataset: Directly supports your framework's multi-format requirement — evaluate how well the agents parse and validate CloudFormation, Terraform, and CDK consistently, since the dataset was built specifically to address the lack of benchmarks across multiple IaC formats [arXiv](#)
- Preprocessing Required: Normalize the three formats into a common intermediate representation for cross-format agent evaluation; split by format for per-technology accuracy reporting

3. Dataset Name: TerraGoat (Vulnerable-by-Design Terraform)

- Source: Bridgecrew (Palo Alto Networks), open-source GitHub repository
- URL: <https://github.com/bridgecrewio/terragoat>
- Size: Full multi-cloud Terraform project (AWS, GCP, Azure modules)
- Number of Records: Dozens of intentionally misconfigured resource blocks across services like EC2, S3, IAM, RDS, EKS
- Number of Features: Resource type, misconfiguration category, associated policy/rule ID (when cross-referenced with a scanner like Checkov)
- Data Type: Terraform HCL code (intentionally insecure configurations)
- License: Open-source (check repo LICENSE file; Bridgecrew tooling is Apache-2.0 style, commonly used for benchmarking without restriction)
- Purpose of Using the Dataset: Built to enable DevSecOps teams to design and test misconfiguration-prevention strategies — use it as ground truth to test your security-assessment agent's detection rate on IAM, network, and storage misconfigurations, since it is less about runtime exploitation and more about learning how insecure Terraform looks in review, scanning, and policy enforcement [GitHubProduct-security](#)
- Preprocessing Required: Tag each resource block with its known vulnerability type (cross-reference with Checkov/tfsec rule outputs) to create ground-truth labels for precision/recall evaluation

4. Dataset Name: Runtime Telemetry & Drift Dataset (project-generated)

- Source: Self-collected from Amazon CloudWatch, AWS Config, and deployment logs within your own AWS test environment
- URL: Not applicable — generated internally during project implementation

- Size: Depends on test deployment duration/scale (recommend at least several weeks of continuous logging across multiple stacks)
- Number of Records: One record per resource-state snapshot or drift event
- Number of Features: Resource ID, expected state (from IaC template), observed state (from CloudWatch/Config), timestamp, drift type, cost delta
- Data Type: Structured JSON/time-series logs
- License: Internal/project-owned (no external licensing, but note AWS service terms apply)
- Purpose of Using the Dataset: This is the dataset that makes your "telemetry-guided reasoning" and configuration-drift-detection objectives testable — none of the public benchmarks above include real runtime telemetry, so this fills that specific gap in your framework
- Preprocessing Required: Time-align telemetry with corresponding IaC deployment versions; normalize AWS Config drift events into a schema matching your template definitions; handle missing/incomplete CloudWatch metrics

AWS Services Planning

AWS Service	Purpose
Amazon Route 53	Provides DNS routing to securely direct users to the application.
Amazon CloudFront	Delivers the web application with low latency using content delivery and caching.
AWS WAF	Protects the application from common web attacks and malicious traffic.
Amazon API Gateway	Exposes secure REST APIs for uploading and validating Infrastructure-as-Code (IaC) templates.
Amazon Cognito	Authenticates users such as DevOps engineers and cloud administrators.
AWS IAM	Manages authorization and access control for AWS resources and services.
AWS Lambda	Orchestrates the validation workflow and invokes the multi-agent validation engine.
Amazon Bedrock	Provides Large Language Model (LLM) inference for intelligent reasoning, validation, and recommendation generation.
Amazon ECS / Amazon EKS	Hosts the containerized Multi-Agent Infrastructure-as-Code Validation Engine.
Amazon S3	Stores uploaded IaC templates, validation reports, and generated recommendations.
Amazon DynamoDB	Stores validation history, metadata, and execution status of validation jobs.
Amazon RDS	Stores user accounts, project information, and application-related data.
Amazon CloudWatch	Collects logs, metrics, and runtime telemetry used for deployment validation, configuration drift detection, and monitoring.
Amazon SNS	Sends notifications when Infrastructure-as-Code validation or analysis is completed.